Usability Fundamentals: A One-Hour Guide

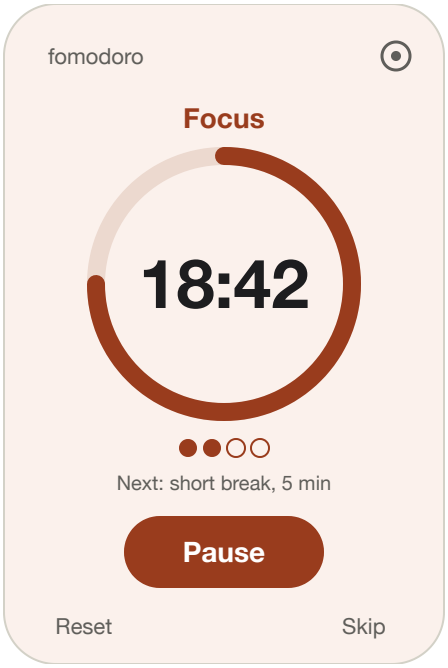
How real people use interfaces, and how to build ones they never have to puzzle over. Written for a frontend developer, with examples from fomodoro. Prepared November 2026.

Where this fits. The *UX Primer* gave you the vocabulary: Nielsen's heuristics, Fitts's and Hick's laws, Norman's principles, progressive disclosure, the HIG, and WCAG. This guide is about the *practice*: what people actually do in front of a screen, and the everyday decisions (a label, a layout, an error message, a five-minute test) that make an interface feel effortless. Examples from our app are in green. Each section ends with a short exercise in a tinted box.

How to use this guide

Read it in one sitting or in ten-minute pieces. Each section lists an estimated reading time, including its exercise. The exercises are short and mostly involve looking at the fomodoro timer (the mockup below, or the real app) and writing two or three sentences. Do them: the whole point of usability is noticing things, and noticing is a skill you only build by doing it.

Section	Topic	Minutes
1. How people really use interfaces	Scanning, satisficing, muddling through	6
2. Self-evident design	Clarity over cleverness, conventions, things that look clickable	6
3. Visual hierarchy and scanning	Gestalt grouping, F- and layer-cake patterns	6
4. Writing for interfaces	Cutting words, microcopy, buttons, errors, empty states	7
5. Navigation and wayfinding	The trunk test, information scent, small apps	6
6. Forms and input	Labels, validation, codes, steppers	7
7. Mobile and touch	Thumbs, targets, interruptions	5
8. Goodwill	What drains trust and what refills it; deceptive patterns	5
9. Usability testing on a budget	Think-aloud, a test script, testing your own app	7
10. Checklist and capstone exercises	Putting it together on fomodoro	5



The fomodoro timer mockup used in the exercises: focus block, 2 of 4 cycle dots filled, running.

1. How people really use interfaces 6 min

Developers know how their software works, so they imagine users reading each screen top to bottom, weighing the options, and choosing the correct one. Decades of observation say otherwise. Steve Krug's *Don't Make Me Think* popularized three blunt facts about real behavior, and every one of them has since been confirmed by eye-tracking and lab research.

People scan; they don't read

On arrival, a person's eyes jump around the screen looking for words and shapes that match what they came to do. They read a word or two, decide whether it's relevant, and jump again. Nielsen Norman Group (NN/g) studies have found for years that people read only a fraction of the words on a typical page, and the share drops as the page gets longer.

Why? Because people are almost always in the middle of something. They have a goal (start a focus block, change a setting) and the interface is in the way of it. Reading is slow; scanning is fast and usually good enough.

In fomodoro: nobody reads the timer screen. They glance at it. In under a second they want to know three things: is it running, how long is left, and is this work or rest. The big digits, the ring, the block name, and the warm or cool tint all answer those questions without a single sentence.

People satisfice

Satisficing is a word coined by economist Herbert Simon: a blend of "satisfy" and "suffice". It means choosing the first option that looks good enough rather than the best one. When someone scans a screen, they don't compare every button. They click the first thing that seems plausible. If it's wrong, they hit Back and try something else, which costs very little on the web.

This is rational. Finding the optimal choice takes effort, guessing wrong is usually cheap, and guessing is sometimes more fun than reading. The design implication is sharp: **the first plausible thing a person sees must actually be the right thing**. If two controls look similar and one is a trap, people will hit the trap about half the time.

In fomodoro: Reset and Skip are both "make this block go away" actions to a scanning eye. Skip moves on to the next block; Reset restarts the current one. If they looked equally prominent and sat side by side, satisficers would hit the wrong one regularly. That's why they're small, labeled in words (not just icons), and placed apart.

People muddle through

Most people never work out how a piece of software is meant to be used. They find a path that works, and they keep using it, even if it's slow or odd. Ask someone to explain how an app they use daily is organized and you'll often get a theory that's partly wrong but works for them.

Two consequences follow. First, if your design depends on users understanding your model, many of them won't. Second, once people find a path that works, they rarely look for a better one. Features they didn't discover in the first week often stay undiscovered forever.

In fomodoro: someone might never open Settings and simply use 25/5/15 for months. That's fine; the defaults should be good enough to muddle through with. But it also means that if Space-to-pause is the only way to pause from the keyboard and it's never mentioned, many Mac users will never learn it. A tooltip on the Pause button ("Pause (Space)") teaches it in passing.

What this implies for the way you build

- **Design for the glance, then the scan, then the read.** The most important information must work at glance speed. Supporting information can wait for a scan. Only rare, deliberate tasks (like setting up 2FA) should require reading.
- **Make the right choice the obvious choice.** If the first plausible option is wrong, that's a design bug, not a user error.
- **Expect half-formed mental models.** Make the interface work even for someone with only a rough idea of how it's organized.
- **Don't rely on instructions.** People skip them. If a screen needs a paragraph of explanation, the screen needs work.

Exercise 1.

1. Look at the timer mockup for two seconds, then look away. Write down everything you remember. Which elements made it into your memory and which didn't? That list is roughly what a scanning user gets.
2. Think of an app you use daily. Name one feature you discovered months late. What would have made you notice it in week one?

2. Self-evident design 6 min

The core idea Krug is known for is that a screen should be self-evident: a typical person should look at it and understand what it is and how to use it without thinking. When that's impossible (some things are genuinely complex), the screen should at least be *self-explanatory*: understandable after a moment's thought, without outside help.

Every small question that pops into a user's head ("Is that a button?", "What does Sync mean here?", "Did that work?") costs a little attention. One question is nothing. Twenty of them make an app feel tiring, even if the user couldn't tell you why.

Clarity beats cleverness

Developers and designers love clever names, novel layouts, and minimalist icons with no labels. Users pay for every one of them in small moments of confusion. A clever label must be decoded; a plain one is just read.

Clever or vague	Plain	Why the plain one wins
Begin your journey	Start focus	Says exactly what will happen.
Recharge	Short break	Matches the word people already use for pomodoro.
Hand-off	Take over here	Describes the result from the user's point of view.
Icon-only gear, bell, cloud, arrows	Gear with an accessible name, plus words where meaning is uncertain	A gear is universal; a cloud with arrows is a guessing game.

A useful test: if you need to explain a label to a friend, the label is wrong. Change the label, not the friend.

Conventions are a gift

A *convention* is a design pattern people have seen so often they no longer notice it: a logo in the top left that goes home, a magnifier that means search, underlined blue text that's a link, a gear for settings, a hamburger for a menu, an X in the corner that closes. Each convention is free knowledge already installed in your users' heads. Using it costs you nothing and saves them a moment of thought.

Developers sometimes resist conventions because they feel unoriginal. Resist that instinct. Innovate where it creates real value for users, and only when your new approach is so clear that it needs no explanation. A good rule: **be conventional by default, and break a convention only if the replacement is at least as obvious and clearly better.**

Platforms have their own conventions, and the Apple HIG from the primer is essentially a catalog of them. iOS users expect a swipe from the left edge to go back, a sheet to slide up from the bottom and be dismissed by dragging down, and toggles (not checkboxes) for on/off settings. Mac users expect keyboard shortcuts, Escape to close dialogs, and Command-comma to open settings.

In fomodoro: the gear in the top-right corner, Space to start and pause (like every media player), Escape to close the settings sheet, and Command-comma on Mac all borrow conventions. A custom "swipe up on the ring to open settings" gesture would be novel, invisible, and would collide with iOS's swipe-up-to-go-home.

Make clickable things look clickable

Flat design took away many of the visual cues that used to say "press me": bevels, shadows, underlines. NN/g research on "flat UI" found that weak signifiers make people slower and less certain, and that they often miss controls entirely. The primer called these cues *signifiers* (Norman's term). Practical rules:

- Primary buttons get a filled shape and strong contrast. Secondary buttons get an outline or a clearly distinct, quieter style, but still look pressable.
- Text links get a distinct color and, in body text, an underline. Don't color plain text with the link color.
- Don't make non-interactive things look interactive: a colored pill that isn't tappable, a card with a shadow that does nothing when pressed.
- Every interactive element gets visible hover, pressed, and keyboard-focus states. The focus ring is not ugly; it is how keyboard users know where they are.

Reduce noise

Self-evidence also depends on what's *not* on screen. Every extra element (a promotional banner, a decorative icon, a second "helpful" hint) competes for the glance. NN/g calls this *visual noise*. If something doesn't help the user's current task, remove it or move it behind a tap.

Exercise 2.

1. On the timer mockup, list every element and mark it: *self-evident* (no thought needed), *self-explanatory* (a moment's thought), or *needs help*. Is anything in the third group? What would fix it?
2. Find one convention fomodoro relies on that a first-time iPhone user would bring with them, and one that only a Mac user would.

3. Visual hierarchy and scanning patterns 6 min

Visual hierarchy is the order in which the eye notices things. A clear hierarchy does a lot of the user's thinking for them: it says what matters most, what belongs together, and what's nested inside what. A flat hierarchy, where everything shouts at the same volume, forces people to read, and we know they won't.

The three rules of a clear hierarchy

1. **Importance shows as prominence.** The more important something is, the bigger, bolder, higher-contrast, or more isolated it should be. You have only a few levels to work with; use them deliberately. The primer's ordering (size, weight, color, space) is a good default toolkit.

2. **Related things look related.** Group items that belong together, and style items that do the same job the same way.
3. **Nesting shows as containment.** If something is part of something else, place it inside or directly under it.

Gestalt grouping, briefly

Gestalt psychology (early twentieth century) describes how the brain groups what it sees before any conscious thought. Four principles cover most interface work:

Principle	What the brain does	In fomodoro
Proximity	Things close together are seen as a group.	The cycle dots and "Next: short break, 5 min" sit together, so they read as one "where am I in the cycle" unit.
Similarity	Things that look alike are assumed to act alike.	Reset and Skip share a quiet style, signaling "secondary actions", distinct from the filled primary button.
Common region	Things inside a shared border or background are a group.	In Settings, durations live in one card and account options in another.
Figure/ground	The eye separates an object from its background.	The ring and digits stand clear of the tinted background; nothing else fights them.

The most common hierarchy bug in developer-built UIs is **uniform spacing**: every element gets the same 16px gap, so nothing groups. Fix it by making the space *between* groups clearly larger than the space *within* them, often double.

How eyes move: F-pattern, layer-cake, and spotted

NN/g's eye-tracking studies identified a handful of scanning patterns on text-heavy pages:

- **F-pattern.** On walls of unformatted text, people read the first line or two, then a shorter line further down, then scan down the left edge. The result looks like an F on the heatmap. This is a symptom of poor formatting, not a goal: it means people are missing most of the content.
- **Layer-cake pattern.** When content has clear headings, people read the headings and skip the body until they find a heading that matches their goal. This is the healthiest pattern; good headings make it possible.
- **Spotted pattern.** People hunt for something with a particular look, like a number, a link, or a bold word, and skip everything else.
- **Commitment pattern.** Highly motivated readers do read everything. It's rare, and you can't design for it.

The lesson: front-load. Put the most meaningful words at the start of headings, labels, list items, and sentences, because the first two or three words are often all that get read. "Notifications: off" scans better than "You currently have notifications turned off".

A single-screen app like fomodoro has no paragraphs to F-scan, but the same instinct applies in miniature. The eye lands on the largest, highest-contrast thing first (the digits), then moves to nearby supporting elements. If you want people to notice something, it has to sit in or near that path.

The squint test

A quick, low-tech check of hierarchy: step back from the screen and squint (or blur a screenshot). The shapes that still stand out are what people will notice first. If the most prominent blob isn't the most important thing, the hierarchy is wrong.

> **Developer tip.** Hierarchy should live in your design tokens, not in ad hoc styles. A short type scale (for example 12, 14, 17, 22, 34, 64px) and a spacing scale (4, 8, 12, 16, 24, 32, 48px) make it hard to accidentally create seven slightly different heading sizes.

Exercise 3.

1. Blur or squint at the timer mockup. Rank the top four shapes by prominence. Does the ranking match what matters most when the timer is running? Would it change when the timer is paused?
2. Sketch the Settings screen as you imagine it. Draw boxes for groups. Which Gestalt principle are you using to separate "durations" from "account"?

4. Writing for interfaces 7 min

Most of an interface is words. Labels, buttons, headings, hints, errors, and confirmations make up what's called *microcopy*, and the discipline of writing it is *UX writing*. For a frontend developer this matters because you often write the first draft of every string in the app, and that draft frequently ships.

Cut words, then cut more

The advice "omit needless words" comes from Strunk and White's *The Elements of Style*, and Krug applied it hard to interfaces. The reasoning ties back to Section 1: people scan, so every extra word dilutes the useful ones and pushes them out of the glance. Two kinds of text are the usual offenders:

- **Happy talk.** Welcoming, self-congratulatory filler. "Welcome to fomodoro, the beautiful timer that helps you achieve your goals!" Nobody reads it, and it pushes the useful content down.
- **Instructions.** Text that explains how to use the screen. Users skip it. If a screen needs instructions, try first to redesign the screen so they're unnecessary. If some instruction survives, cut it to the bone and place it exactly where it's needed.

A practical method: write the string, then delete every word you can without losing meaning. Then try deleting one more and see if it still works.

Before

"Please enter the 6-digit verification code that we have just sent to your email address in the field below."

After

"Enter the code we sent to sam@example.com."

The second version is shorter and better: it shows the actual address (so people can catch a typo), and the six boxes already say "6 digits".

Use the user's words

Write in the vocabulary of the person using the app, not the vocabulary of the codebase. Your code may have `session`, `leaseHolder`, and `blockType: "SHORT"`. The user has "focus", "this device", and "short break". Jargon leaks in through error messages most often, because they're written by developers at the moment something breaks. Keep a short glossary (the domain-modeling notes in the repo are a good home for it) and use exactly those terms in the UI.

Buttons

- **Say what happens.** Start with a verb and name the result: "Send code", "Take over here", "Start focus". Avoid "OK", "Submit", and "Yes" wherever the context allows something specific.
- **Make dialog buttons stand alone.** A person who reads only the buttons should still choose correctly. "Discard changes" and "Keep editing" beat "Yes" and "No".
- **Match the trigger.** If a link says "Install app", the sheet it opens should be titled about installing, not "Add to Home Screen" with no connection.
- **Keep labels stable.** A button that changes its label (Start to Pause) is fine when it's a clear state toggle. A button that changes meaning without changing label is a trap.

Error messages

A good error message answers three questions in plain words: **what happened, why (if it helps), and what to do now**. It appears next to the problem, never blames the user, and preserves their work. NN/g's error-message guidelines add: make it visible (not just a red border), and don't use error styling for things that aren't errors.

Situation	Weak	Better
Expired code	Error 401: invalid token	That code has expired. **Send a new code**
Wrong code	Invalid input	That code doesn't match. Check the latest email, or send a new one.
Offline during sync	Network request failed	You're offline. Your timer keeps running and will sync when you reconnect.
Notifications blocked	Permission denied	Notifications are off for fomodoro in iPhone Settings. **How to turn them on**

In fomodoro: the offline message is also a reassurance. The worst fear when a sync error appears is "did I lose my session?", so answer it directly.

Empty, first-run, and success states

Every view has states besides "full of data": empty, loading, partial, error, and success. Developers usually build the full state first and leave the others as afterthoughts. Users meet the empty state first. A

good empty state says what will appear here and how to get it, with the action right there.

- **Empty:** a history view with no sessions yet: "Finished focus blocks will show up here." No guilt, no confetti.
- **Loading:** show the layout (a skeleton) rather than a lonely spinner, and avoid layout jumps when the content arrives.
- **Success:** confirm quietly and in proportion. "Settings saved" as a brief toast, or no message at all if the change is visible immediately.

Tone

Tone is how the words feel. Fomodoro's tone is calm: no exclamation marks in errors, no streak guilt, no "You broke your streak!" A useful rule from UX writing practice is to adjust tone to the user's likely emotion. A completed focus block can be warm ("Nice work. Break time."). A failed sign-in should be neutral and helpful, never cute.

Exercise 4.

1. Write the microcopy for the moment a focus block ends while the app is open: a heading (3 words or fewer) and one button label. Then write the version for when the user is signed out and their next block is a long break.
2. Rewrite this string in fewer than half the words: "Your timer is currently running on another one of your devices. Would you like to transfer control of the timer to this device instead?"

5. Navigation and wayfinding 6 min

When people use a website or app, they're finding their way through a space they can't see. There's no sense of scale, direction, or location like in a physical building. Navigation is what replaces those senses. Krug framed web navigation as wayfinding, much like signage in a shopping mall: it tells you what's here, where you are, and how to get where you want to go.

What navigation has to do

- **Show what's here.** The visible navigation is a table of contents for the product.
- **Show where you are.** A highlighted tab, a page title, a breadcrumb.
- **Show how to go back or home.** Users get lost; a reliable route home is a safety net.
- **Show how to use it.** Consistent placement teaches people where to look next time.

The trunk test

Krug's *trunk test* is a memorable way to check navigation. Imagine being dropped blindfolded into the trunk of a car, driven around, and let out on a random page or screen. Without scrolling or reading closely, can you answer these quickly?

1. What app or site is this?
2. What screen am I on?

3. What are the main sections or things I can do?
4. What can I do right here?
5. Where am I relative to the rest (and how do I get back)?

Most websites fail at least one. The usual culprit is screen titles: pages with no visible title, or a title that doesn't match the link you clicked to get there.

Information scent

Information foraging theory, developed by Peter Pirolli and Stuart Card at Xerox PARC and popularized by NN/g, says people navigate like animals following a scent. At each step they look at the available links and ask "which one smells most like what I want?" If the scent is strong, they click confidently. If it's weak (vague labels like "Resources", "More", or "Solutions"), they hesitate, guess, or give up.

You strengthen scent by:

- Using specific, user-language labels on links, menu items, and buttons.
- Making the destination match the promise: the screen you land on should use the same words as the link.
- Showing a little preview of what's inside (a current value next to a setting, for example "Focus: 25 min").

Wayfinding in a one-screen app

Fomodoro is mostly a single screen, so it's tempting to think navigation doesn't apply. It does, in smaller form. The "where am I?" question becomes "what state am I in?", and the user needs to answer it at a glance.

Trunk-test question	Fomodoro version	What answers it
What site is this?	Which app or window is this?	Name in the tab title and app icon; on Mac, a recognizable mini-window.
What page am I on?	What block am I in?	"Focus" or "Short break", plus the warm or cool tint (never tint alone).
What are the major sections?	What else is there?	The gear (settings) and, when signed out, a quiet "Sign in" link.
What can I do here?	What's the main action?	One filled Start/Pause button.
Where am I in the whole?	Where am I in the cycle?	Cycle dots (2 of 4) and "Next: short break, 5 min".

Settings and sign-in are the places fomodoro does have navigation. The rules there are ordinary: a clear title on every sheet ("Settings", "Sign in"), a consistent close or back control in the conventional spot, and the same title as the link that opened it. Keep the depth shallow: one level of settings beats nested sub-screens for a personal app.

Cross-device control is a wayfinding problem too. "Timer running on iPhone. Take over here?" tells the Mac user where the session lives right now and what they can do about it, which is exactly what a "you are here" map does.

Exercise 5.

1. Run the trunk test on the timer mockup as if you'd never seen it. Which of the five questions takes longest to answer? Now imagine the screen is *paused* on a short break. Does anything get harder?
2. Write the labels for three settings rows so that each shows its current value (information scent). Example format: "Focus length: 25 min".

6. Forms and input 7 min

Forms are where users do work for you, and where they abandon flows. Every field is a small cost. Good form design reduces the number of fields, makes each one easy to fill correctly, and catches mistakes kindly. Much of this is backed by large-scale research from the Baymard Institute (on checkout forms) and NN/g.

Ask for less

The best field is one you don't need. Before adding an input, ask: can we infer this, default it, or ask later? Fomodoro's sign-in asks for one thing, an email address, and nothing else. No name, no password, no "confirm email".

Labels and layout

- **Visible labels above fields.** Top-aligned labels are fastest to scan and work at every screen width.
- **Never use placeholder text as the only label.** It disappears when typing starts, has weak contrast, and is often mistaken for a pre-filled value. Placeholders are for examples, at most.
- **One column.** Multi-column forms cause people to skip fields or read in the wrong order.
- **Mark the exceptions.** If most fields are required, mark the optional ones, not the other way round.
- **Size fields to their content.** A short box for a short answer hints at what's expected.

Help the device help the user

HTML input attributes are some of the highest-leverage usability tools a frontend developer has, because they summon the right keyboard and autofill for free.

Attribute	What it does	In fomodoro
`type="email"`	Email keyboard with @ key on phones; basic format check.	Sign-in email field.
`autocomplete="email"`	Lets the browser offer the saved address.	One tap to fill on iPhone.

`inputmode="numeric"`	Number pad without the quirks of `type="number"` (no spinner, no scroll-to-change).	The 6-digit code field.
`autocomplete="one-time-code"`	iOS and Android offer the code from Messages or Mail above the keyboard.	The code field; the user taps the suggestion instead of switching apps.
`enterkeyhint`	Labels the Return key ("next", "go", "send").	"send" on the email field.

The 6-digit code field, done well

One-time codes are a good case study because so many apps get them slightly wrong.

- **Use one real input**, even if it's styled to look like six boxes. Six separate inputs break paste, autofill, and screen readers, and make deleting painful.
- **Allow paste** of the whole code, including with spaces or a dash, and strip those characters. WCAG 2.2's Accessible Authentication criterion (in the primer) makes this a requirement, not a nicety.
- **Submit automatically** when the sixth digit arrives, but keep a visible button for people who expect one.
- **Show the email address** the code went to, with a "Change email" link for typos.
- **Make resending easy**, with a short cooldown shown as plain text ("Send a new code in 20 s") rather than a disabled button with no explanation.
- **On error, keep the digits** so the user can see what they typed, and focus the field again.

Validation: when and how

Timing matters as much as wording. NN/g and Baymard both find that validating too early annoys people (red errors while they're still typing an email), and validating too late (only after submit) forces them to hunt back through the form.

- **Validate a field when the user leaves it** (on blur), not on every keystroke.
- **Once a field is in error, re-check on every keystroke** so the error clears the moment it's fixed.
- **Show errors next to the field**, in text (not just color), and link them with `aria-describedby` so screen readers announce them.
- **On submit with errors,** move focus to the first problem and summarize at the top if the form is long.

Prevent errors with the right control

The primer's error-prevention heuristic shows up most clearly in control choice. When the set of valid values is small and bounded, don't use a free text field.

- **Steppers** (minus / value / plus) for small numeric ranges like durations. Focus length from 5 to 90 minutes in 5-minute steps: typing "2500" is impossible. Make the step buttons big enough to tap, support press-and-hold to repeat, and let keyboard users use arrow keys.

- **Toggles** for settings that take effect immediately (sound on/off). **Checkboxes** for choices confirmed later by a button.
- **Segmented controls** for 2 to 5 mutually exclusive options ("Long break every 3 / 4 / 5").
- **Sensible defaults** everywhere. Most users never change them, so the defaults *are* the design for most people.

Save automatically when you can

For settings in a personal app, an explicit Save button is often unnecessary friction and a source of lost changes. Applying changes immediately, with a visible result, is simpler. If a change is risky (turning on 2FA, signing out other devices), confirm that specific action instead.

Exercise 6.

1. List every attribute you'd put on the sign-in email `<input>` and the code `<input>`. Then list three ways a user could end up typing the wrong code, and how the design responds to each.
2. Should "Long break every N blocks" be a stepper, a segmented control, or a text field? Defend your choice in one sentence.

7. Mobile and touch 5 min

Everything above applies on phones, but phones add three pressures: a small screen, an imprecise pointer (a fingertip), and a user who is often distracted or moving. Mobile usability is mostly about respecting those three facts.

Thumbs and reach

Steven Hoober's field observations found that most people hold their phone one-handed much of the time and tap with a thumb. On a large phone, the bottom-center of the screen is easy to reach, while the top corners need a hand shift. This is Fitts's law (from the primer) applied to a body: distance includes how far the thumb has to stretch.

- Put the most frequent action in the easy zone, low and central.
- Put rare or risky actions further away, where an accidental tap is unlikely.
- Don't put frequent actions in the top corners; that's where iOS puts things you rarely need, like Settings-style navigation.

In fomodoro: Start/Pause is large and low; the gear is top-right, rarely needed; Reset and Skip sit low but off to the sides, separated from the primary button.

Target size and spacing

Apple recommends at least 44 by 44 points for touch targets. WCAG 2.2 sets a minimum of 24 by 24 CSS pixels (or enough spacing to compensate). Treat 44 as your real target. The tappable area can be bigger than the visible shape: a small "Skip" label can sit inside a much larger invisible hit area. Spacing

between targets matters as much as size, because a miss that hits nothing is harmless and a miss that hits a neighbor is not.

No hover, and gestures are invisible

- **Hover doesn't exist on touch.** Anything shown only on hover (tooltips, hidden buttons) is unavailable on a phone. Tooltips can teach shortcuts on the Mac, but nothing essential may live there.
- **Gestures have no signifier.** A swipe that does something important is undiscoverable unless there's also a visible control. Use gestures as shortcuts for actions that also have a button.
- **Don't fight system gestures.** Edge swipes, swipe-up-to-home, and pull-down for notifications belong to iOS.

Interruptions and glanceability

Mobile use is fragmented: people look at the phone for a few seconds, put it down, and come back. Design for re-entry. When someone returns, the screen should tell them instantly where things stand, which is why fomodoro's state must be readable at a glance from arm's length.

- Preserve state across app switches, locks, and reloads. A PWA that resets the timer when iOS evicts it from memory has broken the user's trust.
- Keep text large enough to read at arm's length, and respect the user's Dynamic Type or browser text size.
- Respect `prefers-reduced-motion`: a gently animating ring is fine, a pulsing screen is not for everyone.

PWA-specific friction on iPhone

Installing a PWA on iPhone requires the Share menu and "Add to Home Screen", which most people have never used. This is a case where a short instruction is justified, because the platform gives you no install button. Keep it to numbered steps with the real icons, shown at the moment the user asks to install, and dismissible. Push notifications on iOS only work after installation, so the "turn on notifications" flow should explain that in one line and link straight to the install sheet.

Exercise 7.

1. Hold your phone one-handed and imagine the timer mockup at full size. Which controls can your thumb reach without shifting your grip? Is there anything you'd move?
2. Write the three steps of the iPhone install sheet, using no more than 8 words per step.

8. Goodwill 5 min

Krug describes users as arriving with a *reservoir of goodwill*: a supply of patience and trust that each interaction can drain or refill. When it runs dry, people leave, or they stay but resent the product. The level differs by person and by day; someone in a hurry starts with less. This framing is useful because it makes small annoyances add up in your head the way they add up for users.

What drains it

- **Hiding what people want.** Burying a "sign out" or "delete account" option; making cancellation harder than sign-up.
- **Making people do work the app could do.** Requiring a specific phone-number format, losing form data on an error, asking for information you already have.
- **Interruptions.** Pop-ups on arrival, notification prompts before the user has seen any value, "rate this app" mid-task.
- **Looking careless.** Layout jumps, broken states, typos, slow responses.
- **Guilt and pressure.** Streak-shaming, nagging, and fake urgency.

What refills it

- **Make the main thing easy.** Know what people come to do and make it effortless. Opening the app and pressing Start, with no sign-in required.
- **Be upfront.** Tell people what they'll need before they start, and what something will cost (time, data, permissions).
- **Save steps.** Remember settings, prefill what's known, offer the code from Mail.
- **Anticipate questions.** "Your timer keeps running offline" answers the worry before it forms.
- **Make recovery easy.** Undo instead of confirm dialogs where possible; forgiving inputs; errors that tell you the fix.
- **Apologize when you fail.** A plain "Sorry, sync failed. We'll retry automatically" beats silence.

Deceptive patterns

Deceptive patterns (once called dark patterns, a term coined by Harry Brignull) are design choices that trick people into doing things they didn't intend. They often raise a metric in the short term while emptying the goodwill reservoir, and regulators in the EU and US increasingly treat them as unlawful.

Pattern	What it looks like	Fomodoro's stance
Confirmshaming	Opt-out link worded to shame: "No thanks, I don't care about focus".	Neutral declines: "Not now".
Nagging	Repeatedly asking for the same permission after a no.	Ask for notifications once, in context; offer it again only from Settings.
Roach motel	Easy to get in, hard to get out.	Sign out and delete account are plain, findable Settings rows.
Fake urgency	Countdown timers or "only 2 left" pressure.	The only countdown is the one you started.
Guilt loops	Streaks that punish a missed day.	No streaks; history is neutral.
Preselection	Opt-ins pre-ticked in the business's favor.	No marketing email checkbox at all.

Exercise 8.

1. Walk through a first visit to fomodoro, from opening the URL to finishing one focus block. List every moment that could drain goodwill (even slightly) and every moment that refills it.
2. The notification permission prompt: when exactly should fomodoro ask? Write the one-line explanation shown just before the system prompt.

9. Usability testing on a budget 7 min

Everything so far is educated guessing. Usability testing replaces guesses with evidence: you watch a real person try to do real tasks with your interface and note where they struggle. It's the single most effective usability practice, and it's far cheaper than most developers assume.

Why a few users are enough

Jakob Nielsen and Tom Landauer's research found that small tests uncover a large share of usability problems, with steeply diminishing returns: about five users find most of the major problems a given test can find. The practical takeaway is not "five is a magic number" but **test small and often**. Three users this month, fix the problems, then three more next month, beats one big study per year. Krug made the same case for do-it-yourself testing: a morning a month, a few participants, and a debrief the same day.

The kind of testing meant here is *qualitative*: you're looking for problems and their causes, not measuring success rates. You don't need statistics, a lab, or recruits who match your exact target audience. For an app as general as a timer, almost anyone who uses a phone will do.

The think-aloud method

The participant performs tasks while saying out loud what they're looking at, thinking, and expecting. NN/g calls thinking aloud the single most valuable usability tool, because it shows you *why* people get stuck, not just that they did. Your job as facilitator is mostly to stay quiet.

- Give tasks as goals, not instructions: "You want shorter breaks. Set that up," not "Open Settings and change the short break stepper."
- Don't help, don't explain, don't defend the design. When they ask "Should I click this?", answer "What would you do if I weren't here?"
- Prompt neutrally when they go quiet: "What are you thinking now?"
- Watch what they do more than what they say. Opinions ("I'd like it blue") are weak data; behavior ("tapped Skip when they meant Reset") is strong data.

A simple test script

> **Fomodoro test script (about 20 minutes)**
>
> **Intro (2 min):** "Thanks for helping. I'm testing the app, not you; you can't do anything wrong. Please think out loud as you go. I may not answer questions, because I want to see what happens without me. You can stop any time. Is it okay if I record the screen and audio?"
>
> **First impression (1 min):** Show the timer screen, signed out and idle. "Without touching anything: what is this, and what do you think you can do here?"
>
> **Tasks (12 min), one at a time, on a card:**
>
> 1. "You want to focus on some work right now. Get started."
> 2. "Your phone rings. Put things on hold, then carry on."
> 3. "You've finished the task early and want your break now."
> 4. "You'd prefer 50-minute focus blocks. Set that up." (Watch for: noticing that sign-in is needed, and whether that's clear.)
> 5. "Put this app on your home screen so it's easy to find."
>
> **Wrap-up (3 min):** "What was the most confusing part? Anything you expected that wasn't there?" Then thank them.

What to record

Screen and voice recordings are useful (QuickTime on a Mac can capture an iPhone screen over a cable), but your notes matter most. For each task, note:

Field	Example
Task and outcome	Task 3: completed with difficulty.
What happened	Tapped Reset first, timer restarted at 25:00, then found Skip.
What they said	"I thought reset would end it."
Severity	Medium: recoverable, but lost progress.
Idea	Make Reset undoable for a few seconds; consider a skip icon with a forward arrow.

Afterwards, list the problems, rank them by severity and frequency, and fix the worst few first. A common trap is to respond to problems by adding things (a tooltip, a hint, an onboarding tour). The better fix is usually to remove or reword something so the problem can't happen.

Testing your own app as a solo developer

Fomodoro has one primary user: you. That changes the method but not the principle. You are the worst possible tester of your own interface, because you know how it's supposed to work. You can't un-know it. So combine three sources:

- **Dogfood deliberately.** Use the app for real work every day. Keep a friction log, a running note (one line each) whenever something makes you hesitate, misclick, or mutter. Date each entry. Hesitations count, not just failures. Review the log weekly and turn repeated items into issues.
- **Borrow fresh eyes.** Hand your phone to a friend or family member for ten minutes with two or three tasks from the script above. One person who has never seen the app will find things a hundred days of your own use won't. Do it again after each significant change.
- **Change your context.** Use the app in conditions you don't usually test: one-handed while walking, in bright sunlight, with VoiceOver on, with system text at the largest size, on a slow connection, signed out on a fresh device. Each condition is a different "user".

Also run a quick heuristic review with the primer's ten heuristics before each release, and the trunk test on any new screen. These are no substitute for watching a person, but they're cheap and catch the obvious.

> **Make it a habit.** A reasonable cadence for a personal project: friction log always on; a five-minute self-review with the checklist in Section 10 for every UI pull request; one hallway test with a real person per month or per major feature.

Exercise 9.

1. Write one more task for the test script that checks cross-device handoff. Phrase it as a goal, without using the words "take over" or "handoff".
2. Start a friction log today. Use fomodoro for two focus blocks and write down at least three entries, however small.

10. Checklist and capstone exercises 5 min

Use this checklist when you review a screen or a UI pull request. It complements the primer's heuristics table: the heuristics tell you *what* to aim for, and this list is *how to check it quickly*.

Area	Check
Glance	Can someone tell what this screen is for, and its current state, in two seconds?
Primary action	Is there one obvious main action? Is the first plausible choice the right one?
Conventions	Are standard patterns (icons, gestures, shortcuts, positions) used the standard way?
Signifiers	Does everything clickable look clickable, and nothing else? Are focus states visible?
Hierarchy	Does the squint test show the most important thing first? Is spacing between groups larger than within?

Words	Have needless words been cut? Are labels in user language? Do buttons name their result?
States	Are empty, loading, error, offline, and success states designed, not defaulted?
Errors	Do messages say what happened and what to do, next to the problem, without blame?
Wayfinding	Does the screen pass the trunk test? Does each title match the link that led here?
Forms	Visible labels, correct `type`/`inputmode`/`autocomplete`, paste allowed, validation on blur?
Touch	Targets at least 44pt, frequent actions within thumb reach, nothing essential on hover or gesture only?
Goodwill	No nagging, guilt, fake urgency, or hidden exits? Is state preserved across interruptions?
Accessibility	Keyboard-operable, screen-reader names and live regions, contrast, not color alone? (See the primer's WCAG section.)
Evidence	Has at least one person other than the author tried it?

Accessibility is usability

Krug's later editions make the point that accessibility isn't a separate project: most changes that help people with disabilities make the product easier for everyone. Clear labels help screen-reader users and scanners. Large targets help people with tremors and people on a bumpy bus. Captions and text alternatives help deaf users and people in a quiet office. Treat the accessibility row of the checklist as part of every review, not a final pass.

Capstone exercises.

1. **Full review.** Pick one fomodoro screen (timer, Settings, sign-in, or the install sheet) and run the whole checklist. Write down every "no" and a one-line fix for each.
2. **The handoff prompt.** Design the "Timer running on iPhone. Take over here?" moment on the Mac. Decide: where it appears, its exact words, its buttons and their labels, what happens if the user ignores it, and how a screen reader announces it.
3. **Signed-out to signed-in.** A signed-out user taps the gear and tries to change focus length. Write the microcopy that explains why sign-in is needed, without making signed-out mode feel second-class.
4. **Hallway test.** Run tasks 1 to 3 from the Section 9 script with one person this week. Record what happened using the five fields in the table. Fix the worst problem you find.

Where to go next

This guide covered the subject area in its own way, drawing on Krug, NN/g, Baymard, Gestalt psychology, and platform guidelines. For the original and the most enjoyable version of these ideas, **buy Steve Krug's *Don't Make Me Think, Revisited*** (3rd edition, New Riders). It's short, funny, illustrated, and worth reading in full; this guide is not a substitute for it. His follow-up, *Rocket Surgery Made Easy*, is a practical handbook for running the do-it-yourself usability tests from Section 9.

- **NN/g articles** (nngroup.com/articles): search "F-shaped pattern", "information scent", "why you only need to test with 5 users", "thinking aloud", "error message guidelines", "empty states".
- **Baymard Institute** (baymard.com/blog): research-based form and checkout usability.
- **Kinneret Yifrah, *Microcopy: The Complete Guide***: practical UX writing.
- **Harry Brignull, *Deceptive Patterns*** (deceptive.design): catalog and case studies.
- **Steven Hoober's work on mobile touch** (search "how do users really hold mobile devices").
- **web.dev "Sign-in form best practices"** and **"SMS OTP form best practices"**: the code-field details from Section 6.

An original guide prepared for personal learning. It summarizes widely known usability ideas in its own words and attributes them where they originated; it does not reproduce or abridge any book.